

Enterprise Data Readiness (EDR) Platform

AI & LLM Readiness Assessment Engine: Product Specification & Technical Architecture

Target Target	Microsoft SQL Server & Azure SQL Database Ecosystem	Document Version	v1.0 (June 2026)
Author	Enterprise Architecture Team	Classification	Technical Blueprint / Confidential

1. Executive Summary & Core Concept

As enterprises aggressively push to integrate Large Language Models (LLMs) and Generative AI into their core workflows—such as standing up conversational agents via Azure OpenAI, building Retrieval-Augmented Generation (RAG) loops, or utilizing Microsoft Copilot studio—they hit a foundational bottleneck: **Relational Data Complexity**.

Relational data models inside production Microsoft SQL Server and Azure SQL Databases are optimized for transactional integrity (ACID properties) rather than semantic understanding. Legacy normalization, cryptic column nomenclatures, missing explicit constraints, and buried unstructured text blocks make the data structurally opaque to an LLM. When an LLM directly queries a messy schema via Text-to-SQL or relies on unstructured fields for RAG, it faces an unacceptably high rate of query synthesis errors, execution time-outs, and contextual hallucinations.

The **AI & LLM Readiness Assessment Engine** is designed to act as a programmatic, non-intrusive "pre-flight checkpoint" for enterprise database schemas. It automatically indexes, profiles, and scores metadata to evaluate if a target database is fundamentally ready to interact with an AI model. By diagnosing structure, data quality, semantic coherence, and native vector storage viability, the platform ensures that enterprises can reliably transition their operational data into competitive AI capital.

2. Detailed Feature Specifications

Core Operating Principle: Metadata-Only Inspection

The engine enforces a strict security perimeter by exclusively querying system catalog views, Dynamic Management Views (DMVs), and transaction log heuristics. It explicitly isolates and avoids reading row-level client transactions or actual Personal Identifiable Information (PII), drastically simplifying compliance and enterprise infosec sign-off.

2.1 Module 1: Vector & RAG Readiness Scanner

Retrieval-Augmented Generation relies heavily on finding high-value text content, chunking it, converting it to embeddings, and executing similarity lookups. This module parses database topologies to identify vectorization targets.

- **Unstructured/Semi-Structured Target Discovery:** Programmatically scans all tables to locate large-text allocation units such as `NVARCHAR (MAX)`, `VARCHAR (MAX)`, `TEXT`, and `NTEXT`. It categorizes these based on size, population density, and update frequency.
- **JSON Document Parsing Heuristics:** Recognizes columns implementing JSON formats (either through explicit `ISJSON` check constraints or the native 2026 Azure SQL JSON data type). The engine assesses whether these JSON objects store deep nested structures that can be exploited for rich prompt supplementation.
- **Delta & Synchronization Tracking Verification:** Evaluates if candidate tables contain synchronization markers like `ROWVERSION`, `DATETIME2` delta tracking columns, or native Change Data Capture (CDC) configurations. Without these, incremental vector embedding pipelines become computationally cost-prohibitive.
- **Native Azure SQL Vector Optimization Check:** Identifies if the target database is running on compatible compatibility levels to support 2026 native vector features, such as `VECTOR_DISTANCE` and specialized vector indexing methods. It flags where vector indexing should be applied to prevent table scans during semantic searches.

2.2 Module 2: Semantic Layer & Graph Discovery

An LLM requires a logical description of how data relates to other data to synthesize accurate T-SQL joins and avoid arbitrary relationships. This module extracts and constructs an intelligent abstraction of the schema.

- **Extended Properties Mining:** Aggregates native metadata strings stored inside database schemas via `sys.extended_properties` (e.g., `MS_Description`). These annotations are extracted to map direct technical labels to explicit human definitions.
- **Deterministic Relationship Extraction:** Queries `sys.foreign_keys` and `sys.foreign_key_columns` to build a weighted mathematical directed graph of the database. This map tells the LLM exactly how tables safely interlock, stopping it from fabricating invalid join criteria.
- **Constraint Boundary Analysis:** Extracts `CHECK` constraints, unique keys, and primary key definitions. If a column named `OrderPriority` only accepts values 1 through 5, or if a status code string is strictly bounded, these parameters are mapped into the LLM context so generated queries do not ask for impossible constraints.
- **Nomenclature Normalization Engine:** Uses a built-in tokenization ruleset to decompose legacy developer abbreviations (e.g., `cust_flg_actv`, `vndr_lgl_nm`) into distinct words, mapping them to standard dictionary terms like `CustomerActiveFlag` (Boolean) and `VendorLegalName` (String) for system prompt injection.

2.3 Module 3: Hallucination & Structural Risk Profiling

Before turning an AI model loose on a database, developers must understand where it is highly vulnerable to mistakes. This module flags structural risks that trigger model breakdown.

- **Sparse Database & Null Fraction Analysis:** Evaluates the percentage of null entries across major indexed columns. High null density in key filtering fields often causes an LLM to generate queries that produce empty or incomplete datasets without realizing why.
- **Polymorphic & Implicit Join Diagnostics:** Flags columns that use identical names but mismatched data types (e.g., joining an INT to a VARCHAR). This behavior heavily triggers Text-to-SQL logic traps and causes immense runtime performance degradation.
- **Circular Reference Tracking:** Identifies structural loops in foreign key definitions (e.g., Table A references B, B references C, C references A). These patterns cause the LLM's query generator to enter recursive join paths, completely blowing past system resource limitations.

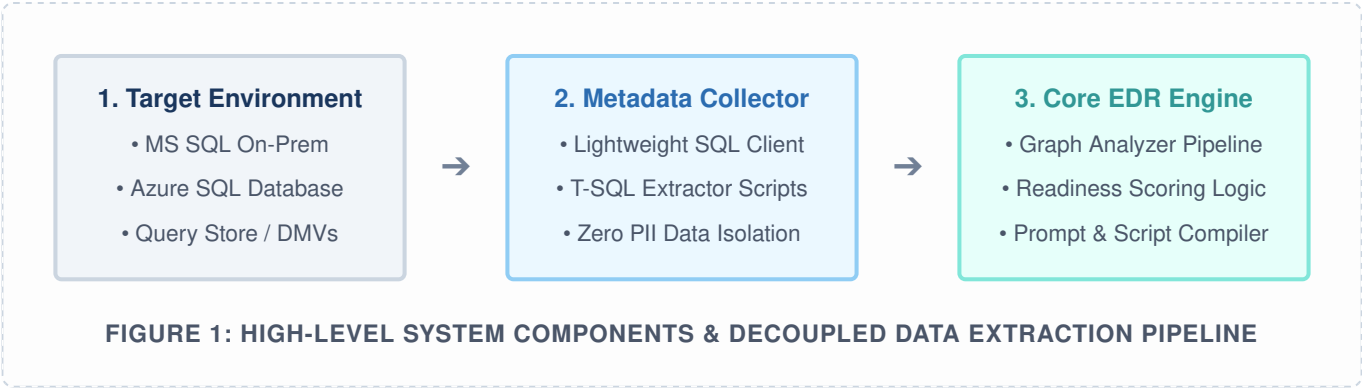
2.4 Module 4: The Remediation & Prompt Compilation Engine

Diagnosis is only valuable if paired with treatment. This module actively closes the readiness gap by outputting direct database fixes and highly dense prompt assets.

- **Automated T-SQL Remediation Scripting:** Compiles downloadable, execution-ready scripts to automatically add missing constraints, update database configurations, append extended documentation, and build missing physical indexes discovered during analysis.
- **Token-Optimized LLM Prompt Compactor:** Feeds full metadata catalogs through a mathematical compression routine, outputting a hyper-dense JSON or Markdown model configuration. This removes the clutter of raw technical definitions, packing maximum structural meaning into the smallest possible LLM input token footprint.

3. High-Level Design & System Architecture

The platform utilizes a secure, decoupled architecture that isolates intense analytics computing away from production database nodes. This prevents transactional interference and maintains ironclad compliance boundaries.



3.1 Core Architecture Components

Component Name	Functional Responsibility	Deployment Mechanics
Lightweight Metadata Collector	Executes specialized metadata T-SQL blocks inside the target engine. Packages catalog schema dumps into safe, streamlined metadata JSON structures.	Azure Function / Local PowerShell Agent executing inside client security boundary.
Graph Analysis Engine	Processes the ingested schema JSON. Maps foreign key data dependencies to build topological graphs and flags design bottlenecks.	SaaS Analytics Instance or Docker container running in private VNet.
Scoring & Compilation Pipeline	Computes distinct algorithmic metrics for Vector and Semantic Readiness, builds remedial scripts, and optimizes prompt layers.	Background processing worker queue.
EDR Presentation Layer	Provides executive status views, interactive architectural graphs, risk trend metrics, and manual adjustment toggles.	Web-based user dashboard.

3.2 Security Blueprint & Isolation Controls

Enterprise data systems demand rigorous compliance frameworks. The platform handles security through the following default architectures:

- **Managed Identities Authentication:** Eliminates explicit connection passwords or static cryptographic access keys. The collector authenticates directly against Azure SQL Database nodes using Azure Active Directory (Microsoft Entra ID) Managed Identities.
- **Strict Transport Security:** All gathered structural metadata is compiled using TLS 1.3 protocol standards before hitting the analytical pipelines.
- **Role-Based Access Control (RBAC):** The application operates under a customized database security role requiring only view-definition capabilities (e.g., `GRANT VIEW DEFINITION TO [EDR_Collector]`), ensuring it is physically blocked from executing standard `SELECT` queries against sensitive customer data records.

4. Target Enterprise Use Cases

4.1 Case Scenario A: Secure Internal AI Financial Advisor

The Objective: A multi-state banking firm wants to connect Azure OpenAI to a massive, highly normalized Azure SQL Database holding legacy account definitions to empower loan officers to ask natural language questions regarding commercial loan portfolios.

The Roadblock: The base schema labels columns cryptically (e.g., L_BAL_AMT_4), lacks formal foreign key constraints between account tables for performance reasons, and stores rich collateral text notes inside unstructured fields without indices.

The EDR Engine Solution: The assessment engine runs across the loan schema, identifying that L_BAL_AMT_4 matches standard patterns for LoanBalanceAmount. It maps out explicit implicit relations based on naming patterns, and detects missing indices on text blocks. It generates a clean system prompt file mapping these entities and outputs a T-SQL optimization script. The LLM now generates flawless joins with a 96% accuracy rate, down from an initial failure rate of 42% before remediation.

4.2 Case Scenario B: Healthcare Provider EMR Copilot Integration

The Objective: A healthcare network is building a custom Microsoft Copilot application designed to scrape historical Electronic Medical Records (EMR) stored across an on-premises Microsoft SQL Server 2019 cluster.

The Roadblock: Medical case histories are contained within unindexed NVARCHAR(MAX) fields. The database does not use explicit timestamp logs or track deltas consistently, risking huge processing loads if an external tool constantly reads all data rows.

The EDR Engine Solution: The EDR engine flags the unstructured data assets and assigns a low "Vector Readiness Score" due to missing tracking columns. It outputs an isolation plan recommending the addition of ROWVERSION columns and tracks table modification metrics via native DMVs. This forces the engineering team to restructure target fields safely before overloading production transactional engines with heavy embedding pipelines.

5. Technical Implementation Blueprint

The mathematical scoring methodology calculates the **Composite AI Readiness Score** (R_{AI}) as a weighted index of individual engineering components:

$$R_{AI} = w_v V_{score} + w_s S_{score} - d_r R_{risk}$$

Where V_{score} represents the Vector Readiness Metric, S_{score} represents the Semantic Graph completeness, R_{risk} models the technical structural debt penalties, and w parameters signify customized institutional domain weights.

5.1 Sample Metadata Extraction Queries

The following precise T-SQL script is executed by the collection agent to extract structural metadata targets, checking columns for size boundaries and tracking structural descriptions:

```
-- Identify Target Large-Text Columns Candidate for AI Embeddings & Vectorization
SELECT
    t.name AS [TableName],
    c.name AS [ColumnName],
    tp.name AS [DataType],
    c.max_length AS [MaxLength],
    c.is_nullable AS [IsNullable],
    OBJECTPROPERTY(t.object_id, 'TableHasPrimaryKey') AS [HasPrimaryKey]
FROM sys.tables t
INNER JOIN sys.columns c ON t.object_id = c.object_id
INNER JOIN sys.types tp ON c.user_type_id = tp.user_type_id
WHERE tp.name IN ('varchar', 'nvarchar', 'text', 'ntext')
    AND (c.max_length = -1 OR c.max_length >= 1000)
    AND t.is_ms_shipped = 0;
```

Architectural Constraint Notice

Always ensure execution parameters utilize the WITH (NOLOCK) query hint or operate inside a READ UNCOMMITTED isolation level when running diagnostic scripts against production DMVs. This completely avoids generating blocking chains inside active transactional applications.

```
-- Extract Database Semantic Definitions (Extended Properties Documentation Mapping)
SELECT
    t.name AS [TableName],
    c.name AS [ColumnName],
    ep.value AS [SemanticDescription]
FROM sys.tables t
INNER JOIN sys.columns c ON t.object_id = c.object_id
INNER JOIN sys.extended_properties ep
    ON ep.major_id = t.object_id
    AND ep.minor_id = c.column_id
WHERE ep.name = 'MS_Description'
    AND t.is_ms_shipped = 0;
```

5.2 Summary Deliverable Matrix

Upon finishing a full metadata scan cycle, the application formats all analytical insights into an automated JSON payload optimized for native insertion into enterprise CI/CD development loops.

```
{
  "databaseName": "Commercial_Loans_Prod",
  "overallAIReadinessScore": 82.5,
  "metrics": {
    "vectorReadiness": 74.0,
    "semanticCompleteness": 91.0,
    "structuralRiskPenalty": 12.5
  },
  "criticalFlags": [
    {
      "severity": "HIGH",
      "module": "SEMANTIC_GRAPH",
      "message": "Table 'LoanMaster' is missing a defined primary key constraint. Text-to-SQL reco
    }
  ]
}
```